

# Prose or Structure?

## Evaluating Security-Claim Generation in LLM-Based Code Analysis

**Sebastian Grünewald**

Diskussionsgrundlage für Dr. Andreas Ekelhart · 3. Oktober 2026

### 1 Idee und Ziel

Wie sollten Sprachmodelle die Ergebnisse einer Sicherheitsanalyse ausgeben, damit diese anschließend zuverlässig weiterverarbeitet werden können? Diese Arbeit vergleicht zwei Wege vom selben Quellcodekontext zu strukturierten Sicherheitsbehauptungen (*Claims*):

**P — über einen Report:** Code → Fließtextbericht → extrahierte Claims

**D — direkt strukturiert:** Code → direkt erzeugte Claims

Beide Wege sollen dasselbe Claimformat verwenden. Verglichen werden die Fundierung im bereitgestellten Code, die Abdeckung relevanter Aussagen sowie der Umgang mit Voraussetzungen und Unsicherheit. Nur bei P kommt die Treue zum ursprünglichen Report als zusätzliche Bewertungsgröße hinzu.

Ein Beispiel verdeutlicht das Problem: Aus „Bei aktivierter Exportfunktion könnten angemeldete Benutzer fremde Dateien lesen“ darf nicht ohne Weiteres „Jeder Benutzer kann fremde Dateien lesen“ werden. Die zweite Aussage lässt eine Bedingung weg und behauptet mehr Gewissheit. Umgekehrt kann eine fachlich falsche Aussage korrekt aus einem Report extrahiert sein. **Bedeutungstreue und tatsächliche Richtigkeit sind deshalb getrennte Fragen.**

Ziel ist ein überschaubares empirisches Paper über die *Erzeugung und Darstellung* von Security-Claims. Eine vollständige automatische Verifikationsplattform wird dafür zunächst nicht entwickelt.

### 2 Forschungsstand und vorläufige Abgrenzung

Die Grundidee, Aussagen zu zerlegen und einzeln zu prüfen, ist bereits erforscht. **DnDScore**, **DecompScore** und **VeriScore** untersuchen Zerlegung, Kontextbehandlung beziehungsweise überprüfbare Aussagen und deren Bewertung [1, 2, 3]. Im Softwarebereich zerlegt **LLMSAN** generierte Datenflusspfade in prüfbare Programmeigenschaften; **GPTAid** gewinnt Sicherheitsregeln unmittelbar aus API-Code und validiert sie dynamisch [4, 5]. **VERGE** und **EviGuard** verbinden strukturierte Claims mit unterschiedlichen Prüfverfahren [6, 7].

Der mögliche Beitrag liegt daher **nicht** in einer erstmaligen Claim-Zerlegung, einer neuen allgemeinen Claim-Sprache oder der bloßen Kombination von Verifiern. Untersucht werden soll vielmehr, welchen Unterschied der Report-Zwischenschritt bei einer *identischen Codegrundlage und einem gemeinsamen Zielschema* macht.

**Vorläufige Forschungslücke:** In den bisher betrachteten Arbeiten wurde kein unmittelbar entsprechender kontrollierter Vergleich identifiziert, der beide Wege anhand von Code-Fundierung, relevanter Abdeckung und erhaltenen Voraussetzungen bewertet. Diese Abgrenzung muss vor der Hauptstudie durch eine gezielte Prüfung der engsten Vorarbeiten weiter abgesichert werden. Eine Überlegenheit der direkten Route wird nicht vorausgesetzt.

### 3 Forschungsfragen

**RQ1 — Was muss ein Claim enthalten?** Welche Kontextangaben und Beziehungen sind erforderlich, damit Security-Claims eindeutig interpretiert und in passende Prüfaufgaben überführt werden können?

*Methode und Ergebnis:* Literatur und manuelle Kodierung von Entwicklungsfällen liefern ein begründetes Claimprofil samt Codebook und Grenzfällen.

**RQ2 — Welcher Erzeugungsweg liefert welche Qualität?** Wie unterscheiden sich P und D hinsichtlich fachlicher Fundierung, relevanter Abdeckung, angemessener Voraussetzungen und Aufwand?

*Methode und Ergebnis:* Gepaarter Vergleich mit unabhängigen Referenzinformationen. Ergebnis sind Qualitäts- und Kostenunterschiede samt Unsicherheit, kein vorab festgelegter Sieger.

**RQ3 — Welche Fehler wirken sich auf die Prüfung aus?** Welche Darstellungsfehler verändern die daraus abgeleitete Prüfaufgabe, und helfen explizite Kontextfelder, diese Fehler zu reduzieren?

*Methode und Ergebnis:* Fehleranalyse und eine kleine Ablationsstudie auf einem Teilbestand. Ergebnis ist eine Einordnung relevanter Fehler und ihrer Folgen; kein neuer allgemeiner Verifier.

### 4 Studiendesign und Claimformat

#### Daten und fairer Vergleich

Ausgangspunkt sind reproduzierbare Java-Fälle aus **Vul4J** mit Patches und Proof-of-Vulnerability-Tests [8]. Zunächst werden zwei bis drei passende Schwachstellenklassen gewählt. JSPWiki bleibt Entwicklungsfall; andere Fälle werden für die Evaluation zurückgehalten. Reparierte Varianten kontrollieren die konkrete Schwachstelle, sind aber nicht pauschal „sichere Programme“.

P und D erhalten identische Quellcode- und Konfigurationsdateien, aber keine Patches, PoVs, Advisories oder Benchmark-Labels. Bei P sieht der Extraktor nur den Report und führt keine zusätzliche Codeanalyse durch. Modellversion, Prompt, Kontext, Wiederholungen, Tokenbudget und Formatvorgaben werden dokumentiert.

Da P zwei Aufrufe benötigt und D zunächst einen, werden komplette Erzeugungswege verglichen. Eine zusätzliche D-Variante mit einem Überarbeitungsschritt sowie Kosten- und Laufzeitauswertungen dienen als Budgetkontrolle. Der Vergleich isoliert dadurch nicht automatisch den reinen Effekt des Ausgabeformats.

#### Ein kleines gemeinsames Claimprofil

CAE und SACM liefern die begriffliche Grundlage für Claims, Argumente und Evidenzbezüge [9, 10]. Für den Pilot genügt ein einfaches strukturiertes Format mit folgenden Feldgruppen:

| Feldgruppe               | Inhalt                                                                                                     |
|--------------------------|------------------------------------------------------------------------------------------------------------|
| Aussage                  | Claim-ID, Proposition, vorläufiger Claim-Typ.                                                              |
| Bedingungen und Umfang   | Akteur/Rechte, Voraussetzungen, Negation, Modalität, betroffene Version oder Konfiguration.                |
| Herkunft und Beziehungen | Codebezüge; Report-Textstelle bei P; explizit benannte verbundene Claims.                                  |
| Prüfbezug                | Benötigte Evidenz oder Prüfaufgabe, getrennt von der behaupteten Aussage und bereits vorliegenden Belegen. |

Annahmen sind keine bestätigten Systemzustände. Extrahierte Aussagen, ergänzender Kontext und abgeleitete Prüfaufgaben bleiben getrennt. Vollständige SACM-Implementierung und Lean-Formalisierung sind nicht vorgesehen.

## 5 Referenzbewertung und Auswertung

Es werden zwei Referenzen getrennt erstellt: Für P werden die tatsächlich im Report enthaltenen Aussagen und Bedingungen annotiert. Für beide Wege wird unabhängig davon festgehalten, welche relevanten Aussagen das bereitgestellte Codepaket rechtfertigt und welche Fragen offenbleiben. Ein echter CVE oder ein erfolgreicher PoV bestätigt nicht automatisch jede Aussage eines Reports.

Mindestens ein Teilbestand soll von zwei Personen unabhängig und möglichst ohne Kenntnis der Erzeugungsroute annotiert werden. Unterschiedliche Bewertungen werden vor der gemeinsamen Klärung aufbewahrt. Ein zweites LLM kann unterstützen, bildet aber nicht allein die Referenz. Maßgeblich ist semantische Übereinstimmung, nicht identischer Wortlaut oder eine feste Anzahl von Claims.

| Kriterium               | Operationalisierung                                                                                                                    |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Fundierung im Code      | Anteil der durch den bereitgestellten Kontext belegten, widerlegten oder nicht entscheidbaren Aussagen; Unsicherheit separat bewerten. |
| Relevante Abdeckung     | Erfasste Propositionen eines vorab abgegrenzten Referenzbestands; keine Belohnung für bloß viele oder triviale Claims.                 |
| Bedeutungstreue (nur P) | Auslassungen, unbelegte Ergänzungen sowie Änderungen an Bedingungen, Negation, Akteur oder Aussagekraft.                               |
| Prüfbarkeit             | Eindeutige Entitäten, Geltungsbereiche und Voraussetzungen; Übereinstimmung der daraus formulierten Prüfaufgaben.                      |
| Aufwand und Robustheit  | Aufrufe, Tokens, Laufzeit, Kosten, Formatfehler und notwendige Reparaturen.                                                            |

Als Vergleich für die Extraktion in P dienen eine einfache satzweise Aufteilung, ein vorhandener allgemeiner Claim-Extraktionsansatz und die codebookgestützte Variante. D ist eine starke direkte Schema-Prompting-Baseline. Security-Systeme wie LLMSAN werden nur bei tatsächlich kompatiblen Teilaufgaben einbezogen. Kleine Zusatzversuche untersuchen die Wirkung expliziter Kontextfelder in beiden Routen.

Ausgewertet werden gepaarte Unterschiede pro Codefall mit Effektgrößen und Konfidenzintervallen. Claims desselben Falls sind keine unabhängigen Stichproben. Annotationsübereinstimmung, Ausfälle und offene Fälle werden separat berichtet. Ein nicht signifikanter Unterschied gilt nicht automatisch als Gleichwertigkeit.

## 6 Arbeitsstand, nächster Schritt und Umfang

**Vorhanden:** Eine Python-Pipeline, ein LLM-generierter JSPWiki-Report, dessen manuelle Zerlegung in 13 Claims sowie ein erstes Codebook. Der zugehörige Java-PoV wurde auf verwundbarer und reparierter Version reproduziert. Das zeigt die technische Machbarkeit, noch nicht die Qualität der automatischen Claim-Erzeugung oder alle im Report behaupteten Auswirkungen.

**Vorgeschlagener nächster Schritt:** Claimprofil und Annotationsregeln an JSPWiki und wenigen weiteren Fällen präzisieren. Anschließend beide Erzeugungswege auf denselben fünf bis zehn Codefällen pilotieren und Annotationzeit, Fehlerarten und Vergleichbarkeit prüfen. Die Hauptstichprobe und der Zeitplan werden erst anhand dieses Piloten festgelegt; die Pilotzahl ist kein ausreichender Publikationsumfang.

**Möglicher Beitrag:** Ein nachvollziehbares Codebook, ein annotierter Referenzbestand und eine reproduzierbare Vergleichsstudie mit konkreten Befunden zu Informationsverlust, unberechtigten Ergänzungen und Qualitäts-/Aufwandskonflikten. Positive Ergebnisse werden ebenso wenig vorausgesetzt wie zwei Veröffentlichungen.

**Abgrenzung:** Keine vollständige Agentenplattform, kein universeller Security-Scanner und kein eigenes Modelltraining. Eine mögliche Anschlussarbeit untersucht später, welche unabhängige Evidenz welche Claim-Typen tatsächlich rechtfertigt. Für den nächsten Austausch sind vor allem Umfang, zweiter Annotator und ein geeigneter Pilotbestand abzustimmen.

## Ausgewählte Literatur

- [1] M. Wanner, B. Van Durme und M. Dredze (2025). *DnDScore: Decontextualization and Decomposition for Factuality Verification in Long-Form Text Generation*. EMNLP, S. 23609–23626. [doi:10.18653/v1/2025.emnlp-main.1205](https://doi.org/10.18653/v1/2025.emnlp-main.1205).
- [2] M. Wanner, S. Ebner, Z. Jiang, M. Dredze und B. Van Durme (2024). *A Closer Look at Claim Decomposition*. Verwendete Fassung: arXiv:2403.11903. [arxiv.org/abs/2403.11903](https://arxiv.org/abs/2403.11903).
- [3] Y. Song, Y. Kim und M. Iyyer (2024). *VeriScore: Evaluating the Factuality of Verifiable Claims in Long-Form Text Generation*. Findings of EMNLP, S. 9447–9474. [doi:10.18653/v1/2024.findings-emnlp.552](https://doi.org/10.18653/v1/2024.findings-emnlp.552).
- [4] C. Wang, W. Zhang, Z. Su, X. Xu und X. Zhang (2024). *Sanitizing Large Language Models in Bug Detection with Data-Flow*. Findings of EMNLP, S. 3790–3805. [doi:10.18653/v1/2024.findings-emnlp.217](https://doi.org/10.18653/v1/2024.findings-emnlp.217).
- [5] J. Liu, Y. Yang, K. Chen und M. Lin (2025). *Generating API Parameter Security Rules with LLM for API Misuse Detection*. NDSS. [doi:10.14722/ndss.2025.23465](https://doi.org/10.14722/ndss.2025.23465).
- [6] V. Singh, D. Cassel, N. Weir, N. Feng und S. Bayless (2026). *VERGE: Formal Refinement and Guidance Engine for Verifiable LLM Reasoning*. Verwendete Fassung: arXiv:2601.20055v2. [arxiv.org/abs/2601.20055v2](https://arxiv.org/abs/2601.20055v2).
- [7] H. Zhou, H. Lei und M. Yang (2026). *EviGuard: Machine-Verifiable Evidence Grounding for LLM-Based Industrial Incident Reasoning*. Applied Sciences, 16(18), 8925. [doi:10.3390/app16188925](https://doi.org/10.3390/app16188925).
- [8] Q.-C. Bui, R. Scandariato und N. E. Díaz Ferreyra (2022). *Vul4J: A Dataset of Reproducible Java Vulnerabilities Geared Towards the Study of Program Repair Techniques*. MSR, S. 464–468. [doi:10.1145/3524842.3528482](https://doi.org/10.1145/3524842.3528482).
- [9] Adelard (o. J.). *Claims, Arguments and Evidence (CAE)*. Konzeptionelle Dokumentation; abgerufen am 03.10.2026. [adelard.com/asce/cae/](https://adelard.com/asce/cae/).
- [10] Object Management Group (2021). *Structured Assurance Case Metamodel (SACM), Version 2.2*. Normative Spezifikation und Metamodell. [omg.org/spec/SACM/2.2](https://omg.org/spec/SACM/2.2).

**Dokumentstatus.** Vorschlag für ein erstes Paper, nicht das formale Exposé einer gesamten Dissertation. Grundlage sind die Projektnotizen „Prose or Structure?“ und die bisherige Diskussion. Methoden und Umfang sind abzustimmen; eine abgeschlossene Studie oder bestätigte Neuheit wird nicht behauptet.

**Layout.** Vereinfachte Anpassung von „[Template Text – University of Vienna](#)“ (Universität Wien, [CC BY 4.0](#)); ohne Logo und separate Titelseite. Keine offizielle Research-Proposal-Vorlage.